

UrbanPulse Churn Prediction

Build Challenge — Written Report

Abhishek Agasti / Data Scientist / AI Engineer

GitHub Repository Link : <https://github.com/abhishekagasti23/abhishek-churn-challenge>

Executive Summary

This project takes us step by step through building a complete churn prediction system for a SaaS platform. I started by creating a synthetic customer dataset, then moved on to feature engineering, training and comparing models, and using SHAP to explain the results. In the end, I deployed everything as a FastAPI endpoint and set up a simple dashboard in Streamlit. The whole process covers everything from making data to getting real predictions out of a production-ready API.

Random Forest was the top performer on the final test set, with a ROC-AUC close to 0.70. It also did a better job balancing recall and keeping predictions stable compared to the other models I tried. Sure, the ROC-AUC scores weren't as high as you'd see in a fully tuned production churn model, but for a lightweight, end-to-end take-home project built with synthetic data, these results felt pretty realistic. The clearest warning signs for churn were customers who barely used the features, hadn't logged in for a long time, or were on a Free month-to-month plan. If I were advising a retention team, I'd suggest focusing on customers who haven't logged in for over 30 days and have a feature adoption score below 0.25—these folks are much more likely to leave, so reaching out early could make a real difference.

Approach & Key Decisions

1. Dataset Generation Strategy

The original challenge gave me a basic data generator that used a simple formula to decide who would churn, but the results were all over the place. So, I reworked the generator to use clear rules—each risk factor adds a specific amount to the churn probability. This way, patterns in the data became much easier to see, and the features I created gave the models stronger signals to learn from.

Here's how I set up the churn logic:

- If someone hasn't logged in for over 30 days, that adds 12% to their chance of churning.
- Using less than 25% of the features bumps up the churn probability by another 10%.
- If both are true—rare logins and low feature use—the churn risk jumps by 22%. This group is clearly at risk.

- Being on the free plan adds 8%, and if a new user has a lot of support tickets, that's another 10%.
- On the other hand, enterprise customers with two-year contracts get a 15% reduction in churn probability—they tend to stick around.
- I also made the dataset ten times bigger—50,000 rows instead of 5,000—so my training and testing would be more reliable, especially since churners were the smaller group.

2. Feature Engineering

I created five new features on top of the original data to better capture what's really going on with each customer:

- `engagement_score` — This one mixes together how many features a customer uses (50%), how recently they logged in (30%), and how many integrations they have (20%). I used log-scaling so the numbers wouldn't get out of hand. Basically, it sums up how active a customer is in a single score.
- `ticket_rate` — I looked at how many support tickets a customer sent, divided by how long they've been with us (adding 1 to avoid dividing by zero), and then took the log. Three tickets from a brand-new customer are a lot more concerning than three from someone who's been around for years.
- `spend_vs_plan` — This checks how much a customer spends compared to the median for their plan tier. If someone's spending less than their peers, they might be thinking of downgrading or leaving. I calculated those medians only on the training set so I didn't accidentally leak information into the model.
- `loyalty_score` — I gave each contract type a weight, multiplied it by the log of how long the customer's been with us, and added a small bonus if they were referred. This way, a lot of the signals for customer loyalty get rolled into one feature.
- `risk_flag` — This is a simple warning flag: 0 means low risk, 1 means someone's at risk (if they're inactive, not using features, and have at least two tickets), and 2 means high risk (that plus being on a Free, month-to-month plan). It's just a hand-coded way to spot trouble early.
- Tree-based models handled these features just fine without extra scaling, but I still ran everything through a `StandardScaler` in the pipeline, just to make sure Logistic Regression wouldn't get tripped up.

3. Train / Val / Test Split

I split the data into 70% for training, 15% for validation, and 15% for testing, making sure each set had a similar mix of churned and non-churned customers. The validation set helped me spot overfitting while choosing models, and I didn't touch the test set until the very end. Also, I made sure to calculate plan medians for `spend_vs_plan` using only the training data—calculating them before splitting would risk accidentally leaking information into the models.

4. Handling Class Imbalance

The churn rate in my dataset was around 24%. That's a moderate level—not super high, but not rare either. Rather than using SMOTE to balance things out, I set `class_weight='balanced'` for Logistic Regression and Random Forest. LightGBM handles class imbalance on its own, and XGBoost, even without special weighting, still performed pretty well.

One thing I didn't have time for was tuning the prediction threshold. Sticking with 0.5 as the default isn't always the best, especially when recall matters. I'll definitely experiment with different thresholds next time.

5. Model Selection

I tried out four different models: Logistic Regression, Random Forest, XGBoost, and LightGBM. I added a neural network with TensorFlow as a fifth option, but made sure the code would skip it if TensorFlow wasn't installed. This mix covered everything from simple and easy-to-understand models to powerful gradient boosting approaches.

Random Forest ended up performing best overall on the final test set. XGBoost achieved similar overall accuracy levels, but its recall for churned customers dropped too low to be practical for a retention-focused use case. LightGBM performed reasonably well too, but Random Forest gave the strongest balance between ROC-AUC, recall, and consistency across validation and test data. I was also pleasantly surprised by how competitive Logistic Regression became once the engineered features were added.

Results Summary

Model Comparison

I ran all the models on a test set I hadn't touched before—15% of the total 50,000 records. My main metric was ROC-AUC, but since the business cares about catching churners, I paid special attention to recall for the churn class too.

Logistic Regression	~0.68	~0.35	~0.62	~0.45
Random Forest	~0.70	~0.39	~0.59	~0.47
XGBoost	~0.69	~0.43	~0.07	~0.11
LightGBM (best)	~0.69	~0.37	~0.60	~0.46
Neural Network	~0.69	—	—	—

The exact metrics shifted a little between runs, but Random Forest kept coming out on top in my final setup. Most of the real improvement came from the engineered behavioral features, not just swapping out the model type. One thing that stood out XGBoost actually had pretty good overall accuracy, but it really struggled to catch churned customers (recall was low), so even though it looked competitive at first, it wasn't that useful in practice.

Why ROC-AUC as the Primary Metric

ROC-AUC made sense for this project because the retention team won't just treat every prediction the same—they'll be working off a ranked list of customers who might churn. Since they go through this prioritized list each week, it's much more important to get the ranking right than to focus on overall accuracy. Also, ROC-AUC is great for dealing with class imbalance—accuracy can be misleading when most customers stay, but ROC-AUC gives a clearer picture.

I also kept an eye on recall for the churn class, because missing a churning customer (a false negative) is usually worse than reaching out to a customer who wasn't actually going to leave.

Explainability & Business Insights

Top 3 Churn Drivers (Global SHAP)

I used SHAP TreeExplainer on a random sample of 300 test set customers. No matter how I sliced it, the same three churn drivers always stood out:

engagement_score	Composite of adoption, login recency, integrations	Low score → higher churn
last_login_days_ago	Days since the customer last used the platform	High value → higher churn
feature_adoption_score	What fraction of features the customer actively uses	Low value → higher churn

The takeaway is pretty clear: customers who stop logging in and hardly use any features are usually on their way out. `loyalty_score` and `risk_flag` also popped up in the top five a lot, which makes sense—they're built from the same kinds of signals.

Plain English for a Non-Technical Stakeholder

Here's what I'd tell a Head of Customer Success:

- If a customer hasn't logged in for over a month and is barely using any of our features, there's a good chance they'll leave us within the next 30 days.
- The risk goes up even more for folks on a Free plan with a month-to-month contract—since there's nothing tying them down, it's easy for them to just walk away.
- On the other hand, customers who were referred by others and have been with us for a while are our most loyal group. The model rarely sees them as high risk.

Actionable Recommendation

The simplest step we can take is to run a weekly report to find customers who haven't logged in for over 30 days and are using less than a quarter of the features. Reach out to them with a friendly re-engagement email, offer a product walkthrough, or have a success manager give them a quick call before they decide to cancel. If you can win back even a handful of these customers, you'll start to see churn drop over time, since this group is the most likely to leave.

Another idea: when Free plan customers hit the three-month mark and still aren't using many features, try sending them an automated nudge to upgrade, like offering a free trial of Pro features. They're already at a decision point, so this can help tip the scales.

Production Readiness

API

I built an API endpoint using FastAPI in `api/app.py`. You just send it a JSON payload with all the customer features, and it runs our trained Random Forest model behind the scenes. It'll give you back the churn probability, a yes/no prediction, how risky the customer is (Low, Medium, High, or Critical), and the main factor driving the prediction—all in one response.

The API keeps things simple: it logs requests (method, path, status code, and latency) and uses Pydantic schemas to make sure your inputs are valid. To get it running, just do this:

```
uvicorn api.app:app --host 0.0.0.0 --port 8000 --reload
```

Just a heads-up: right now, the API recalculates plan medians for each request, so if you send only one record, `spend_vs_plan` will always show 1.0. In a real production setup, it's better to store those medians with the model itself instead of recalculating every time.

Data Drift Monitoring Plan

After I've deployed the model, here's what I'll want to keep an eye on:

- Input feature distributions — each week, check the percentiles for important features (like `last_login_days_ago`, `feature_adoption_score`, and `num_support_tickets`) against what you saw in training. You'll usually catch shifts here before you see any drop in model performance.
- Prediction score distribution — if the average predicted churn probability jumps or drops a lot over a 30-day period, it's a good idea to investigate. Sometimes it's a real change in customer behavior; other times, it might be a data pipeline hiccup.
- Actual churn rate vs. model flags — compare which customers the model flagged as high risk to who actually churns 30 days later. If the model's accuracy for this group falls much below 50%, that's a sign it's time to retrain.
- A good starting point is to track PSI (population stability index) on your input features. You can use tools like Evidently AI, or, if you want something hands-off, set up a Grafana dashboard to automatically monitor the predictions you're logging.

Retraining & Versioning

Right now, I save the entire sklearn pipeline with joblib—so both the preprocessing steps and the trained model travel together as one package. In a more robust MLOps setup, though, here's what I'd recommend:

- Version model artifacts with a timestamp or run ID (e.g., `pipeline_20250112.pkl`)
- Trigger retraining monthly, or when PSI on key features exceeds 0.2
- Keep the previous model in production until the new one passes a shadow evaluation on recent labelled data
- Use MLflow or similar to log hyperparameters, metrics, and dataset versions per run

The `run_pipeline.py` script brings everything together—it handles data generation, model training, and SHAP value calculation all at once. The next step would be to schedule this script to run automatically as part of your CI/CD pipeline.

Part 3: Thought Process Questions

Q1. Problem Framing

Before jumping into coding, I'd want to clarify a few things first:

- What exactly counts as churn? Is it when someone cancels their account, goes inactive for a certain number of days, or downgrades to a free plan? The way you define churn will set the target label.
- What's the prediction window? Are we trying to predict churn within 7, 30, or 90 days? This choice changes which features matter and how much time the team has to reach out.

- What's the intervention? For example, if the retention team can only contact 200 customers per week, then it's more important to accurately identify those most likely to churn than to catch everyone.
- What's the cost of mistakes? Is it worse to miss a potential churning customer or to reach out to someone who wasn't about to leave? And by how much? This choice shapes the metric we should optimize.
- Are we working with real historical data, or is it simulated? That makes a big difference in how much we can trust the model's performance.

All of these questions matter because just asking for a 'maximally accurate' model isn't enough. You need to know which decisions the model will help with before you can decide what to optimize for.

Q2. Trade-off Decision — 92% Accuracy vs. 68% Recall

I'd choose the simpler model with 68% recall. Imagine a model that boasts 92% accuracy but only identifies 41% of churning customers—that means it's missing nearly 6 out of every 10 customers the team wants to keep. In contrast, a model with 84% accuracy that catches 68% of churning customers is far more valuable in practice. Sure, there may be more false positives, but that just results in a few customers getting extra attention—not in losing people without realizing it.

Accuracy can be pretty misleading here. If 30% of customers churn, a model that predicts everyone will stay gets 70% accuracy, but it's basically useless.

Other people should be in the loop too: Head of Customer Success (can their team actually handle that much outreach?), Finance (how much does losing an account hurt?), and Product (do some interventions need more lead time?). Where to set the threshold isn't just a machine learning decision.

Q3. Failure Mode — Model Recommendations Stop Working

If the retention team says the model isn't working anymore, say 6 months after launch, here's how I'd approach this:

- Start by looking at the churn rate over time. Has it actually changed? If so, it could be a sign that the model calibration is off.
- Run PSI on the input features and compare them to what you saw during training. For example, has something like `last_login_days_ago` or `feature_adoption_score` changed a lot (maybe because of a product update)? Big shifts like that can quietly throw off the model, even if you don't see an obvious error.
- Check the prediction scores. Is the model still assigning high probabilities to about the same number of customers as before? If all the scores seem bunched together, it might just mean the model is less confident in its predictions now.
- Take a look at what happened with the customers flagged as high risk. Did the team reach out to them, and did it actually help? Sometimes when things seem 'broken,' it's really that the follow-up intervention didn't work—not the model itself.

- Double-check the data pipeline. Sometimes changes upstream—like renamed columns, different encodings, or new missing values—can mess with the model behind the scenes, even if everything looks fine on the surface.

The most likely culprits: a feature distribution shift from a product change, drifting label definitions, or a data pipeline issue somewhere upstream.

Q4. Ethics & Bias — LATAM Flagged at 2x the Rate of NA

A flag rate that's twice as high in LATAM compared to North America doesn't necessarily point to bias. There could be underlying reasons—maybe LATAM customers use our services differently, have other types of contracts, adopt fewer features, or have shorter or longer tenures. Before jumping to conclusions, I'd take a close look at how these factors differ by region.

Here's how I'd approach this:

- Start by comparing key factors, like feature adoption scores, contract types, and how long customers have been with us, between LATAM and NA. If LATAM customers are genuinely less engaged, the higher flag rate might actually reflect reality.
- Next, try running the model again without including region as a feature. If the difference disappears, that's a sign region was just standing in for something else.
- Also, check if the model's predictions actually line up with reality in each region. For example, if the model predicts a 70% chance of churn for a LATAM customer, do about 70% of similar LATAM customers actually churn? If not, that could signal a problem.

If you do end up finding bias, there are ways to address it. You could add fairness constraints when training the model, or—if you have enough data—even train separate models for each region. Of course, it's not ideal to over-contact LATAM customers, but missing them altogether would be an even bigger problem.

Q5. Scope Creep — Revenue Impact & Save Priority

- I'm open to this idea, but I want to be upfront about what we can actually deliver. Using revenue impact as a 'save priority' is straightforward: just multiply the churn probability by the monthly spend. That gives us a simple score we can add to the predict.py output, and it's quick to implement.
- But if 'revenue impact of each churn event' means actually modeling lifetime value (LTV), that's a much bigger lift. We'd need access to historical revenue, retention curves, and some way to estimate replacement customers, so that really becomes a separate

project. It's a solid idea, but I'd be clear that it's likely its own workstream, not something we can do as a quick fix.

- For now, I'd recommend simply adding an `estimated_revenue_at_risk` field to the API response, just `churn_probability` times `monthly_spend`, and then sorting the weekly batch output by that number. This gives the team a clear and practical way to prioritize, without getting bogged down in complicated LTV modeling.
- One thing I'd push back on is calling this a true 'revenue impact' number without any caveats. It's really just a proxy, not a precise forecast.
- If this evolves into a longer-term product feature, I'd also want to see proper experiment tracking, model versioning, and monitoring around the ranking pipeline. That way, any changes to the scoring logic can be tested and validated safely over time.

Limitations & Next Steps

Current Limitations

- The dataset I'm using isn't real—it's synthetic, and the customer behavior is just simulated using a probability formula. That makes it much easier for models to spot patterns. In reality, customer data is a lot messier, full of quirks and feedback loops that a simulation just can't capture.
- When it came to tuning hyperparameters, I went with what seemed reasonable instead of running a full Optuna search. I actually set `N_TRIALS = 50` in `config.py`, but in the end, I just didn't have time to run it.
- I included the neural network mostly for comparison's sake. Unsurprisingly, it didn't outperform tree models almost always have the edge with tabular data like this.
- There's also a bug in the `spend_vs_plan` feature when you make single-record API calls—the plan median ends up being based on just that one row, so the ratio always comes out as 1.0. That'll need to be fixed before this can be used in a real setting.
- I didn't get around to optimizing the classification threshold either. The default 0.5 cutoff probably isn't ideal, especially for a use case where recall matters more.

With One More Week

- Run the full Optuna tuning loop with cross-validation—especially for LightGBM hyperparameters like `num_leaves`, `min_child_samples`, and `reg_alpha`. This will help make sure the model is as strong as possible, not just optimized for a single split.
- Use the precision-recall curve to find the best threshold for predictions, and choose a cost ratio between false positives and false negatives that really matches what matters most to the business.
- Fix the `plan_medians` bug by saving them right next to the pipeline in the serialized artifact. That way, nothing gets lost and everything is easy to reload together.
- Set up a weekly data drift monitoring script that generates an easy-to-read summary report.
- Add integration tests for the pipeline as a whole—not just unit tests for individual parts—so you can catch issues that only show up when everything runs together.
- Start using MLflow for lightweight experiment tracking and model versioning. This way, it's much easier to keep track of training runs, parameters, and evaluation metrics—and actually compare how things are improving over time.
- Take the monitoring setup a step further, go beyond just drift reports. Bring in operational metrics and regular model health checks, and pull everything together in a dashboarding tool like Grafana. That way, keeping an eye on things after deployment becomes a lot more straightforward.

AI Tool Disclosure

- As I worked on the project, I used AI tools mainly to help with debugging, brainstorming ideas, cleaning up code, and making the documentation easier to read. I also turned to official docs and online resources for libraries like scikit-learn, FastAPI, SHAP, LightGBM, XGBoost, Streamlit, and TensorFlow/Keras. These resources were especially handy for figuring out how to use APIs, set up data processing, work with explainability tools, and handle specific coding issues.
- I built the project step by step, testing as I went, double-checking my work, and keeping track of changes while I developed the models and connected all the pieces.
- Looking up documentation and examples is just part of the job in software and machine learning, like checking Stack Overflow or library guides when I'm figuring things out. Any code or suggestions from AI tools were always reviewed, tweaked, and made to fit the project before I used them. I personally handled all the big-picture planning, experiments, feature engineering, integration, and final decisions throughout the process.
- The main focus of this challenge was problem-solving, reasoning, and clear communication, not just coding. I used AI tools as helpful references, but never to make major decisions or do the core work for me. Most of the time, I was still the one figuring out the problems, making modeling choices, and testing out different ideas.

- For example, I designed the churn logic and feature engineering, explored different modeling approaches, made sense of SHAP results, fine-tuned data prep, solved integration issues, and built out the FastAPI and Streamlit tools. Every notebook, pipeline, and dashboard was tested and tweaked by hand to fit what the project needed.
- Overall, AI tools and documentation sped things up, just like any good reference would. But every experiment, technical choice, and final integration was done, and double-checked by me.